

Bloque 05 - Pandas: datos tabulares fiables

Propósito

Una empresa no necesita que alguien «limpie un Excel»: necesita poder responder, sin cambiar la respuesta cada vez, cuántos pedidos válidos hubo, qué ingresos representan y qué información falta. En este bloque Leo trabaja como analista de **Nébula**, una aplicación de suscripción que vende complementos. Recibirá exportaciones de pedidos y clientes, construirá una tabla analítica y dejará evidencia de cada decisión.

Pandas es una biblioteca de Python para trabajar con tablas. No decide qué dato es correcto: convierte reglas de negocio explícitas en transformaciones repetibles y comprobaciones que pueden fallar.

Resultado de salida y prerequisites

Al terminar podrás cargar un CSV realista, diagnosticarlo, limpiarlo sin perder el original, combinarlo con clientes y publicar una tabla de ingresos por canal reconciliada con el detalle. Necesitas los conceptos de fila, columna, clave, nulo y Python básico de los bloques 01 y 02.

El caso usa los archivos [pedidos_nebula.csv](#) y [clientes_nebula.csv](#). Son pequeños deliberadamente: permiten inspeccionar cada anomalía antes de automatizarla.

Itinerario

1. Importar y perfilar una tabla
2. Seleccionar, tipar y limpiar sin ocultar pérdidas
3. Transformar, agrupar y reconciliar una métrica
4. Unir tablas y proteger la cardinalidad
5. Contrato de datos, validación y linaje
6. Laboratorio: pipeline de pedidos de Nébula

Práctica

Primero ejecuta el [laboratorio reproducible](#). Después resuelve [la auditoría de pedidos](#) sin mirar la [solución razonada](#).

Documentación primaria

- Pandas: lectura de CSV
- Pandas: merge, join y compare
- Pandas: Copy-on-Write

01 - Importar y perfilar una tabla

Objetivo y prerequisites

Al terminar podrás explicar qué representa una fila de un CSV, cargarla con parámetros deliberados y producir un perfil inicial antes de calcular una métrica. Requiere saber que una tabla tiene filas y columnas.

Del archivo a una tabla en memoria

Imagina un archivo de texto con estas dos líneas:

```
pedido_id;fecha_pedido;importe_bruto
P-1001;2026-06-03;29,90
```

Un **CSV** es un archivo de texto que separa valores; su nombre histórico dice «comma-separated», pero aquí el separador es ;. No lleva una garantía de tipos: 29,90 llega como caracteres, no como dinero. Un **DataFrame** es la tabla que Pandas mantiene en memoria; una **Series** es una sola columna, con un valor por fila y un índice que identifica su posición.

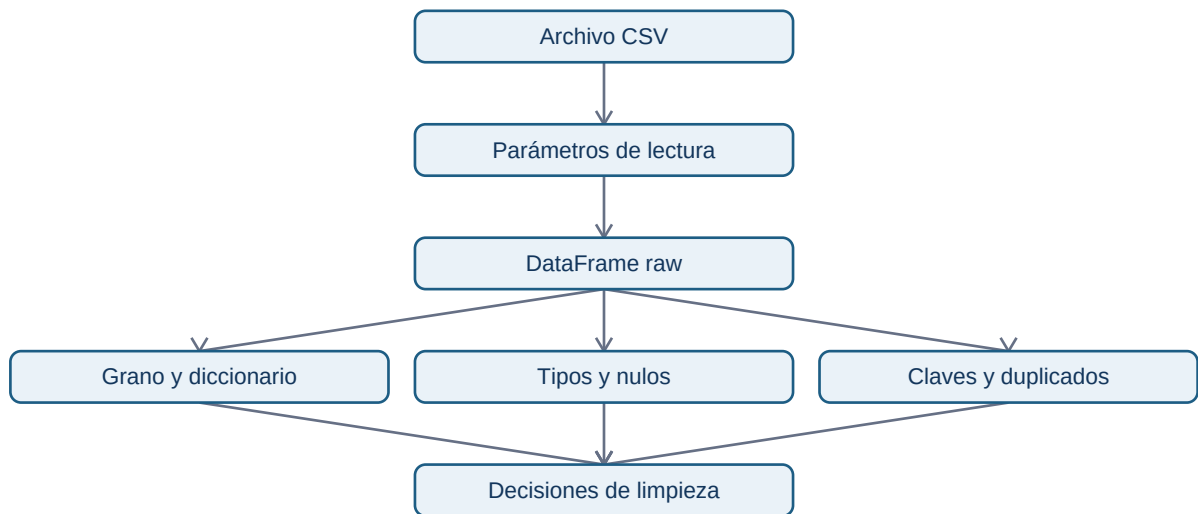
En Nébula una fila de pedidos_nebula.csv pretende representar **un pedido creado**. Esa frase es el *grano*: si una fila fuese una línea de producto, sumar importes o contar pedidos cambiaría de significado.

```
from pathlib import Path
import pandas as pd

datos = Path("datasets/pandas")
pedidos_raw = pd.read_csv(
    datos / "pedidos_nebula.csv",
    sep=";",
    encoding="utf-8",
    na_values=["", "NA", "sin dato"],
    dtype={"pedido_id": "string", "cliente_id": "string", "canal": "string"},
)
```

dtype protege identificadores: un ID no es una cantidad y no se debe convertir en número. Las fechas y los importes se convertirán después, de forma visible, porque primero interesa descubrir qué valores no cumplen el formato.

El diagrama responde a «¿qué debo saber antes de transformar una exportación?»:



La importación no es todavía limpieza. Produce una versión raw que se conserva para poder explicar de dónde salió cualquier fila descartada.

Perfil mínimo que evita errores caros

Antes de preguntar «¿qué canal vende más?» pregunta qué llegó:

```
print(pedidos_raw.head(3))
print(pedidos_raw.shape)
print(pedidos_raw.dtypes)
print(pedidos_raw.isna().sum())
print(pedidos_raw["pedido_id"].duplicated().sum())
print(pedidos_raw["estado"].value_counts(dropna=False))
```

`head()` ofrece ejemplos, no prueba calidad. `shape` permite detectar una carga incompleta. `isna().sum()` cuenta ausencias por columna. `value_counts(dropna=False)` muestra tanto categorías inesperadas como nulos; sin `dropna=False` podríamos no ver que falta un estado.

Un perfil profesional también define un pequeño diccionario: `pedido_id` es la clave esperada, `fecha_pedido` es la fecha de creación en UTC, `importe_bruto` está en EUR y `estado` decide si el pedido entra en ingresos. No basta con que los nombres «suenen bien».

Error habitual y límite

Un error frecuente es usar `pd.read_csv("pedidos.csv")` y continuar porque no dio excepción. Si el fichero usa `;`, Pandas puede construir una sola columna enorme; si usa coma decimal, el importe puede quedar como texto. La carga técnicamente correcta no demuestra que la semántica sea

correcta.

Otra falsa seguridad es hacer `pd.to_datetime(..., errors="coerce")` y asumir que todo se interpretó. En datos no estándar o mezclados es preferible convertir luego con `pd.to_datetime(..., errors="coerce")`, medir los fallos y decidir qué hacer con ellos.

Resumen y comprobación

- CSV describe una forma de separar texto; DataFrame es la tabla en memoria.
- Grano, clave y unidades se declaran antes de agregar.
- El perfil mide lo que llegó; no lo corrige a escondidas.

1. ¿Por qué `pedido_id` debe leerse como texto aunque contenga dígitos?
2. Si aparecen 500 filas en lugar de 50 000, ¿qué comprobarías antes de concluir que hubo menos pedidos?

Sigue con [selección](#), [tipos](#) y [limpieza](#).

02 - Seleccionar, tipar y limpiar sin ocultar pérdidas

Objetivo y prerequisites

Transformarás el extracto de Nébula en una tabla utilizable sin confundir una corrección técnica con una decisión de negocio. Partimos del DataFrame `pedidos_raw` de la lección anterior.

Seleccionar es formular una condición

`pedidos["canal"]` devuelve una Series. `pedidos[["pedido_id", "canal"]]` conserva un DataFrame. Para seleccionar con intención hay dos herramientas: `loc` usa etiquetas y una condición; `iloc` usa posiciones numéricas para inspección, no para reglas de negocio.

```
pedidos = pedidos_raw.copy()
pagados = pedidos.loc[pedidos["estado"].eq("pagado")].copy()
ejemplo = pedidos.iloc[:3, :4]
```

La máscara `pedidos["estado"].eq("pagado")` es una Serie de `True/False`, una respuesta por fila. Antes de filtrar, cuenta los estados. «Pagado» no equivale siempre a «cobrado», «facturado» ni «sin devolución»: aquí es una definición operativa que debe figurar en el contrato.

Tipos, fechas y nulos: convertir sin inventar

Los tipos (dtypes) determinan qué operaciones son válidas. Para Nébula la fecha llega como texto y el importe usa coma decimal:

```
pedidos["fecha_pedido"] = pd.to_datetime(
    pedidos["fecha_pedido"], format="%Y-%m-%d", errors="coerce", utc=True
)
pedidos["importe_bruto"] = pd.to_numeric(
    pedidos["importe_bruto"].str.replace(",", ".", regex=False), errors="coerce"
)
pedidos["descuento"] = pd.to_numeric(
    pedidos["descuento"].str.replace(",", ".", regex=False), errors="coerce"
).fillna(0)
```

`errors="coerce"` convierte una conversión imposible en ausente (NaN o NaT); es una alarma medible, no una reparación. Por ejemplo, una fecha inválida no debe convertirse silenciosamente en la fecha de hoy. Después clasificamos el motivo y conservamos las filas rechazadas:

```
es_valido = (
    pedidos["pedido_id"].notna()
    & pedidos["fecha_pedido"].notna()
    & pedidos["importe_bruto"].ge(0)
    & pedidos["canal"].isin(["web", "app", "partner"])
)
rechazos = pedidos.loc[~es_valido].assign(motivo="regla_basica")
pedidos_validos = pedidos.loc[es_valido].copy()
```

Copias y duplicados tienen significado

`copy()` comunica que el resultado será una tabla independiente. Evita modificar de manera inesperada un subconjunto de `pedidos_raw` y evita depender de comportamientos de vista/copia que han evolucionado en Pandas. La regla práctica: conserva `raw`, crea pasos con nombres y asigna con `.loc` sobre el DataFrame que posees.

Un **duplicado técnico** es una fila idéntica repetida por una exportación. Un **duplicado de negocio** son dos filas que comparten `pedido_id` aunque otro campo cambie; puede representar reintento, corrección o corrupción. No se tratan igual:

```
duplicados_tecnicos = pedidos.duplicated(keep=False)
duplicados_negocio = pedidos.duplicated("pedido_id", keep=False)
print(pedidos.loc[duplicados_negocio].sort_values("pedido_id"))
```

Eliminar con `drop_duplicates("pedido_id")` sin inspección puede quedarse con la primera versión arbitraria. En el laboratorio se conserva la fila más reciente por `fecha_extraccion`, una regla que debe validarse con el área dueña de la fuente.

Resumen y comprobación

- loc expresa reglas con nombres; iloc inspecciona posiciones.
- Convertir con coerce hace visibles los errores; no equivale a aceptar la fila.
- raw, tabla válida y rechazos son artefactos distintos que permiten auditar.

1. ¿Qué información perderías al ejecutar dropna() sobre toda la tabla?
2. ¿Por qué dos filas con el mismo pedido_id requieren una conversación de negocio antes de eliminarlas?

Sigue con [transformación y agregación](#).

03 - Transformar, agrupar y reconciliar una métrica

Objetivo y prerequisites

Construirás ingresos netos por canal sin perder la relación con cada pedido. Requiere una tabla de pedidos válida y saber que el grano actual es un pedido.

Una columna derivada es una regla, no una fórmula suelta

Nébula quiere ingresos netos de pedidos pagados. El contrato acordado es: importe bruto menos descuento, en EUR, por pedido creado en junio; no representa margen ni ingresos tras devoluciones futuras.

```
pedidos_validos = pedidos_validos.assign(
    importe_netos=lambdas tabla: tabla["importe_bruto"] - tabla["descuento"]
)
assert pedidos_validos["importe_netos"].ge(0).all()
```

assign devuelve una nueva tabla y el lambda deja claro que la regla usa las columnas de esa misma tabla. El assert verifica un supuesto; si falla, debemos mirar las filas, no cambiar el umbral para que el programa continúe.

De detalle a resumen sin cambiar el denominador

groupby reúne filas con el mismo valor de una clave de agrupación. Después agg define con nombre qué resume cada columna:

```
por_canal = (
    pedidos_validos.groupby("canal", as_index=False)
    .agg(
```

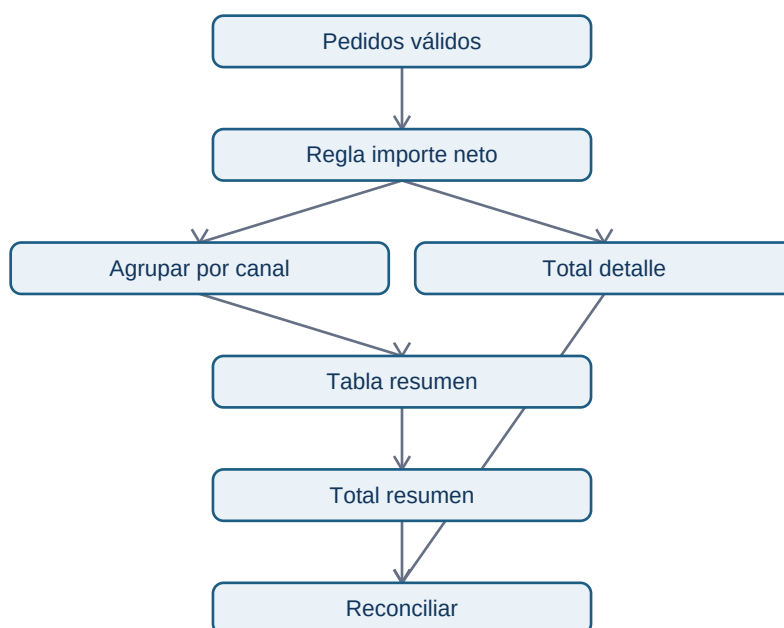
```

    pedidos=("pedido_id", "nunique"),
    ingresos_netos=("importe_netos", "sum"),
    ticket_medio=("importe_netos", "mean"),
)
.sort_values("ingresos_netos", ascending=False)
)

```

nunique cuenta pedidos distintos; count cuenta valores no nulos y size cuenta filas. Si aún hubiese dos registros para un pedido, size inflaría el volumen. El promedio no debe viajar solo: un canal con un único pedido caro puede tener el mayor ticket y aportar poco al total.

Este flujo responde a «¿cómo sé que el resumen no perdió o duplicó dinero?»:



La reconciliación compara dos caminos que deberían coincidir:

```

total_detalle = pedidos_validos["importe_netos"].sum()
total_resumen = por_canal["ingresos_netos"].sum()
assert total_detalle == total_resumen

```

En importes con muchos decimales se usaría `math.isclose`, porque la representación binaria de `float` puede introducir diferencias minúsculas. Para facturación real, la unidad monetaria y el redondeo se acuerdan con finanzas; no se resuelven solo con Python.

Error habitual y pregunta analítica

Agrupar por canal responde una pregunta descriptiva: «¿cómo se distribuyen los ingresos observados?». No demuestra que el canal haya causado la venta: web, app y partner pueden atraer clientes distintos. También puede ocultar país, campaña o periodo; por eso una segunda agrupación

debe añadir una hipótesis, no columnas por costumbre.

Resumen y comprobación

- Una medida derivada lleva definición, unidad, población y límite.
- La función de conteo debe coincidir con el grano.
- Reconciliar es comparar el detalle con el resumen antes de publicar.

1. ¿Cuándo usarías size en lugar de nunique?
2. ¿Por qué la igualdad entre totales no demuestra por sí sola que los estados incluidos sean correctos?

Sigue con [uniones y cardinalidad](#).

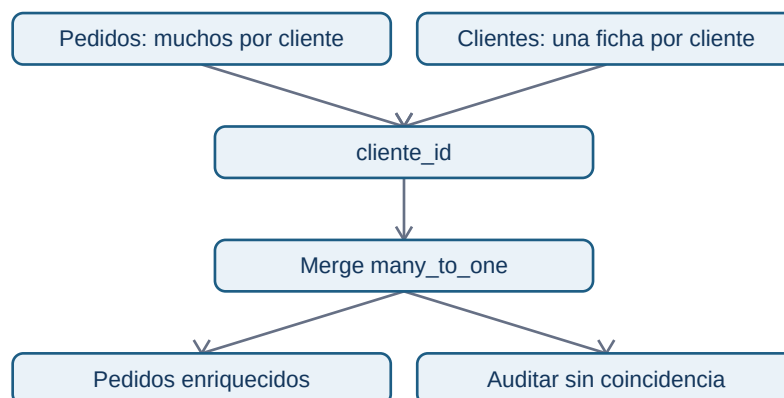
04 - Unir tablas y proteger la cardinalidad

Objetivo y prerequisites

Enriquecerás pedidos con el segmento de cliente sin multiplicar ingresos por accidente. Requiere conocer clave, grano y el resumen de la lección anterior.

La pregunta que precede a merge

pedidos contiene muchos pedidos por `cliente_id`; clientes debe contener una sola ficha vigente por cliente. Por tanto, la relación esperada es **muchos a uno**: muchos pedidos encuentran un cliente. Una unión combina columnas mediante una clave; no es una prueba de que la clave represente a la misma persona en ambos sistemas.



`how="left"` conserva todas las filas del lado izquierdo, importante cuando un pedido no encuentra cliente: ocultarlo convertiría un problema de cobertura en una aparente mejora de calidad.

```
clientes = pd.read_csv(datos / "clientes_nebula.csv", sep=";", dtype="string")
assert clientes["cliente_id"].is_unique

enriquecidos = pedidos_validos.merge(
    clientes[["cliente_id", "segmento", "pais"]],
    on="cliente_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
print(enriquecidos["_merge"].value_counts())
```

`validate` hace fallar el código si `clientes` contiene la misma clave dos veces. `_merge` clasifica el resultado: `both` coincidió, `left_only` no encontró ficha. La elección de excluir `left_only` depende de la métrica: para ingresos de pedidos suele conservarse el pedido y se etiqueta su segmento como desconocido; para analizar segmentación se comunica la cobertura.

Cardinalidades y contraejemplo

- **uno a uno:** una fila de cada lado por clave;
- **uno a muchos:** una cuenta tiene muchos eventos;
- **muchos a uno:** muchos pedidos pertenecen a un cliente;
- **muchos a muchos:** cada clave se repite en ambos lados. Puede ser válido en una tabla puente, pero multiplica combinaciones.

Si el archivo de clientes tuviera dos fichas para C-10, un pedido de 40 EUR podría salir dos veces tras el merge y convertirse falsamente en 80 EUR. La conciliación de la lección 03 debe ejecutarse otra vez después de cada unión que afecte a las filas.

Resumen y comprobación

- Declara cardinalidad y `how` antes de escribir el merge.
 - `validate` protege el supuesto; `indicator` mide cobertura.
 - Una unión correcta en sintaxis puede ser errónea en negocio.
1. ¿Por qué un `inner merge` puede ocultar pedidos importantes?
 2. ¿Qué tabla adicional necesitarías para modelar correctamente una relación muchos a muchos entre pedidos y productos?

Sigue con [contrato](#), [validación](#) y [linaje](#).

05 - Contrato de datos, validación y linaje

Objetivo y prerequisites

Convertirás expectativas sobre los datos en controles y describirás el camino desde un archivo fuente hasta un resultado. Requiere el pipeline de las lecciones anteriores.

El contrato evita que «limpio» sea una opinión

Un **contrato de datos** es un acuerdo comprobable entre quien publica y quien consume una tabla. No tiene que ser una plataforma compleja: para pedidos_nebula basta documentar el grano, clave, columnas, tipos, rangos, categorías permitidas, zona horaria, actualización y propietario.

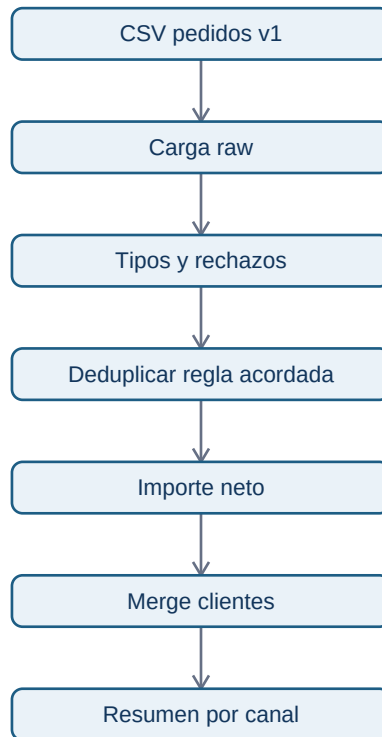
Cada regla se comprueba cerca de la transformación que la necesita:

```
def comprobar_pedidos(tabla: pd.DataFrame) -> None:
    requeridas = {"pedido_id", "cliente_id", "fecha_pedido", "importe_netto", "canal"}
    assert requeridas.issubset(tabla.columns), "Faltan columnas del contrato"
    assert tabla["pedido_id"].is_unique, "Un pedido aparece más de una vez"
    assert tabla["fecha_pedido"].notna().all(), "Hay fechas no interpretables"
    assert tabla["importe_netto"].ge(0).all(), "Hay importes netos negativos"
```

Un `assert` es adecuado para un pipeline educativo o una comprobación interna. En producción debe convertirse en una señal con contexto: número de filas, muestra segura de claves, versión de fuente y decisión de detener, avisar o aislar datos.

Linaje: poder responder «¿de dónde sale este número?»

El **linaje** registra origen, transformaciones y salida. El diagrama responde esa pregunta para ingresos por canal:



Junto a cada ejecución conserva: fecha/hora de extracción, ruta o identificador de versión, conteo de entrada, rechazos por motivo, filas de salida y total reconciliado. No guardes datos personales innecesarios en el registro; IDs y muestras deben tratarse según la política de privacidad.

Límite y resumen

Pasar validaciones no prueba que una métrica sea útil: podría cumplir el contrato y medir la fecha de creación cuando dirección quería fecha de cobro. El contrato aclara y detecta desviaciones; la decisión de negocio sigue necesitando dueño y contexto.

- El contrato define lo que se espera antes de ejecutar código.
- Las validaciones protegen supuestos de grano, tipos y rangos.
- El linaje permite repetir y auditar una cifra.

1. ¿Qué dato mínimo guardarías para explicar por qué cambiaron los ingresos de ayer?
2. ¿Qué regla del contrato no puede inferir Pandas y debe acordarse con negocio?

Aplica el flujo en el [laboratorio integrado](#).

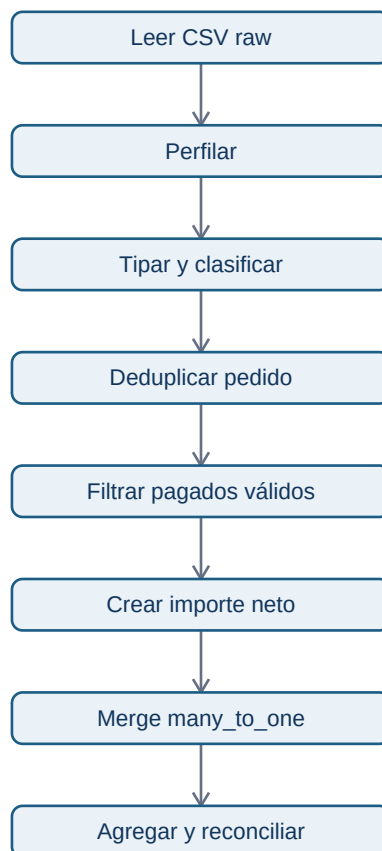
06 - Laboratorio: pipeline de pedidos de Nébula

Objetivo y entrega

Ejecuta [05-pipeline-pedidos-nebula.py](#). El script no descarga nada: lee los dos CSV del repositorio, muestra los perfiles, aísla rechazos, deduplica de forma explícita, calcula ingresos netos, une clientes y reconcilia el resumen.

La pregunta de negocio es: «**en la extracción de junio, ¿cuántos pedidos pagados válidos e ingresos netos observamos por canal, y qué cobertura tiene la segmentación de clientes?**». La respuesta no es margen, LTV ni causalidad de canal.

Secuencia de trabajo



Al ejecutar, revisa especialmente tres decisiones: una fecha inválida se rechaza en vez de inventarse; una actualización duplicada de P-1002 se resuelve por fecha de extracción; un cliente sin ficha se conserva en ingresos y se declara como cobertura incompleta.

Resultados esperados y lectura profesional

El resultado debe imprimir Ingresos por canal, un total de detalle igual al total del resumen y una tabla de both/left_only. Si cambias una regla, por ejemplo incluyes pendientes, no basta con que el script termine: modifica la definición de la métrica, vuelve a conciliar y explica el impacto.

Un pipeline pequeño es ya una entrega profesional si otra persona puede ejecutarlo, entender sus supuestos y detectar qué datos quedaron fuera. La automatización no sustituye a confirmar que pagos, devoluciones y moneda corresponden a la decisión.

Después resuelve [la auditoría de pedidos](#). El bloque 06 utilizará esta tabla trazable para explorar patrones; no convertirá una diferencia entre canales en una explicación causal.